Perfect approach. Let's walk through the **Starter Mini-Challenge** step-by-step with **thorough explanations** and **conceptual examples**, without giving away ready code. The goal is for you to fully understand *what* you're doing and *why*—and solve it on your own.

🧩 Project: **Mergeable Shopping Cart**

This console-based Dart project teaches you how to:

- Work with `Map<String, int>` (a dictionary-like structure)
- Handle merging data structures
- Compare maps
- Use lists for storing history
- Build undo functionality

✅ Step 1: **Create and Define Cart Structure**

What to do:

- Decide how to represent a shopping cart.
- A cart will store item names (as strings) and quantities (as integers).

Example (conceptual):

Imagine two carts:

- Cart A: contains `"apple": 2` and `"banana": 3`
- Cart B: contains `"banana": 1` and `"orange": 5`

Both carts should be structured as Dart `Map`s.

What you'll practice:

- Declaring and using `Map<String, int>`
- Adding entries, updating values
- Iterating through a map

🔁 Step 2: Merge Two Carts

What to do:

- Implement logic to combine two carts into one.
- If an item appears in both carts, its quantity should be summed.
- If it appears only in one cart, it should just be included as is.

Example (conceptual):

Merging the above two carts should result in:

- `"apple": 2`, `"banana": 4`, `"orange": 5`

What you'll practice:

- Copying maps
- Iterating through key–value pairs
- Using map methods like `containsKey`, `update`, or `putIfAbsent`

🧮 Step 3: Compare (Diff) Two Carts

What to do:

- Create a function that compares two carts and finds:
  - Items that are **newly added** (exist only in the second cart)
  - Items that are **removed** (exist only in the first cart)
  - Items with **changed quantities** (exist in both, but quantities differ)

Example (conceptual):

Cart A:

- `"apple": 2`, `"banana": 3`

Cart B:

- `"banana": 5`, `"orange": 1`

Diff Result:

- Added: `"orange": 1`
- Removed: `"apple": 2`

- Changed: `"banana": 3 → 5`

What you'll practice:

- Comparing keys in two maps
- Finding differences between values
- Creating new maps for reporting differences

🧠 Step 4: Track Merge History

What to do:

- Before you merge a new cart, save the current cart's state.
- Store previous versions in a list (as copies of the cart).
- This allows "undo" functionality later.

Example (conceptual):

You start with Cart A, then:

1. Merge Cart B → New state (Cart A1)
2. Merge Cart C → New state (Cart A2)

History list stores:

- [Cart A, Cart A1]

What you'll practice:

- Creating deep copies of maps
- Managing a list of previous states
- Thinking about when to record snapshots

⏪ Step 5: Implement Undo

What to do:

- Allow the user to undo the last merge.
- This means retrieving the most recent previous cart state from the history list and restoring it.

Example (conceptual):

Cart A → Merged into A1
Undo → Revert to original Cart A

What you'll practice:

- Accessing and removing the last element from a list
- Replacing map contents safely
- Defensive programming (checking if history is empty before undoing)

🖥️ Step 6: User Interface in Console

What to do:

- Create a menu with simple options like:
  - View current cart
  - Merge another cart
  - Show diff between current and new cart
  - Undo last merge
  - Exit program

Use `stdin.readLineSync()` to read user choices and react accordingly.

Example (conceptual):

```
Choose an option:
1. View cart
2. Merge another cart
3. Show cart diff
4. Undo last merge
5. Exit
```

What you'll practice:

- Console input/output
- Handling user choices
- Writing clean, reusable functions for each action

🧪 Step 7: **Test Your Program Thoroughly**

What to do:

- Try merging:
  - A cart with overlapping items
  - A cart with completely different items
  - An empty cart
- Try undoing multiple times
- Try comparing two identical carts

What you'll practice:

- Thinking about edge cases
- Building confidence in your solution
- Debugging unexpected behaviors

🎯 Bonus Challenges (Optional for later)

- Add prices per item and calculate total cost.
- Store items as objects (with `name`, `quantity`, `price`, etc.) instead of just strings and ints.
- Save/load cart history to and from a file using `dart:io`.

Let me know when you're ready to begin **Step 1** or if you'd like to recap any of the steps. I can walk you through each individually while keeping everything in your control.